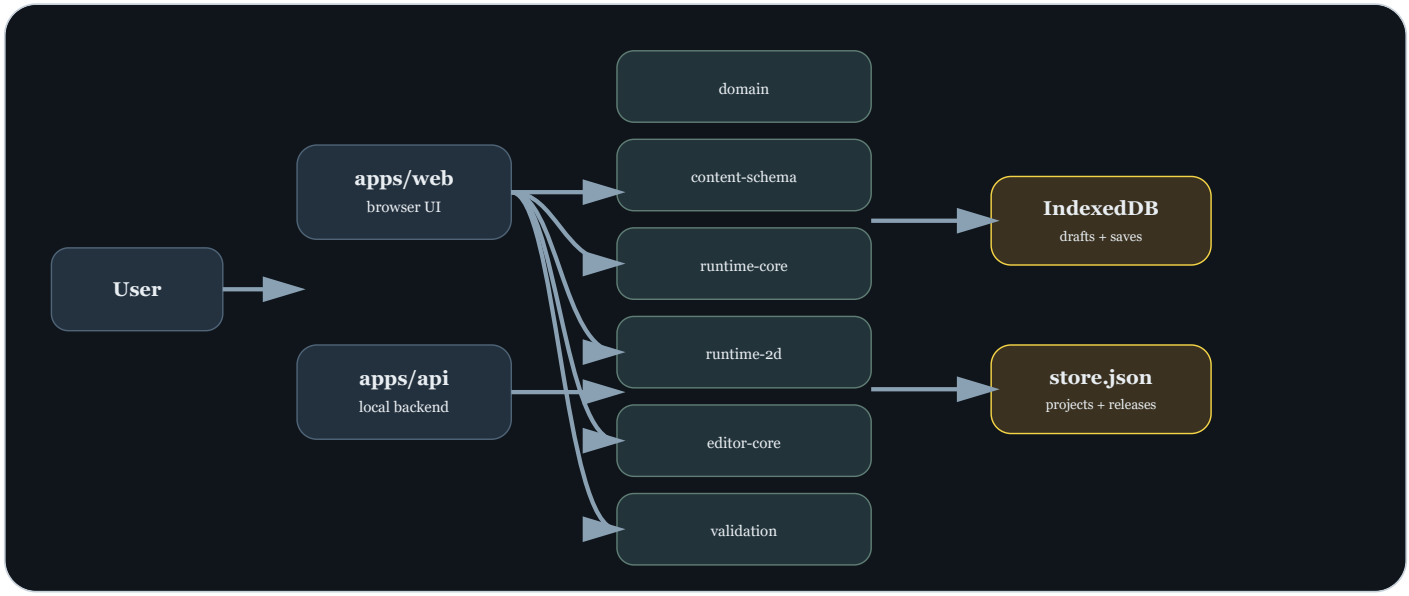
ACS MILESTONE 15

System Reference

A technical map of how ACS fits together, with concrete end-to-end flows for runtime movement, inspection, editor tile painting, entity placement, validation, persistence, and rendering.

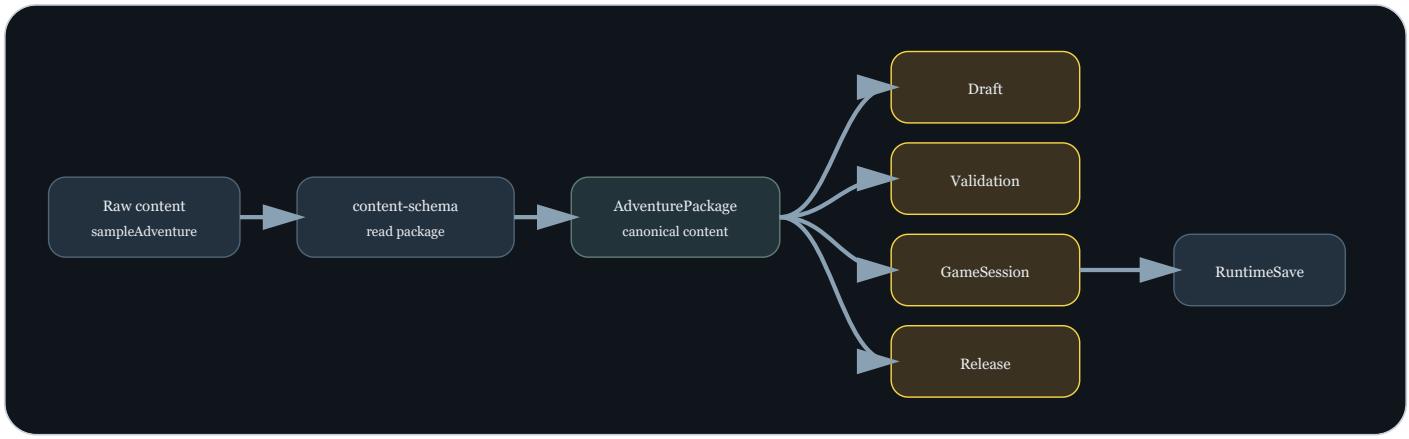
Companion documents: the user guide explains how to use the application; this system reference explains what happens internally. The Markdown source includes Mermaid diagrams, and this PDF includes readable rendered equivalents plus screenshots.

High-Level Architecture



Area	Package or app	Responsibility
Shared vocabulary	`packages/domain`	IDs, maps, entities, triggers, dialogue, actions, and conditions.
Content ingestion	`packages/content-schema`	Reads raw authored content and normalizes it into an `AdventurePackage`.
Simulation	`packages/runtime-core`	Commands, state mutation, triggers, dialogue, enemy turns, snapshots, and events.
Rendering	`packages/runtime-2d`	Draws runtime state to canvas without owning game rules.
Editing	`packages/editor-core`	Pure draft operations such as `setTileAt`, `addEntityInstance`, and `moveEntityInstance`.
Validation	`packages/validation`	Publish-readiness checks shared by browser and API.
Persistence	`packages/persistence`	IndexedDB runtime saves and local editor drafts.
Backend	`apps/api`	Projects, releases, validation endpoint, and file-backed storage.

Data Model Layers



Key distinction: a release is immutable content, while a save is runtime state. A save wraps the engine's existing `RuntimeSnapshot`, so persistence does not invent a second save-game model.

Current Screens

Milestone 15

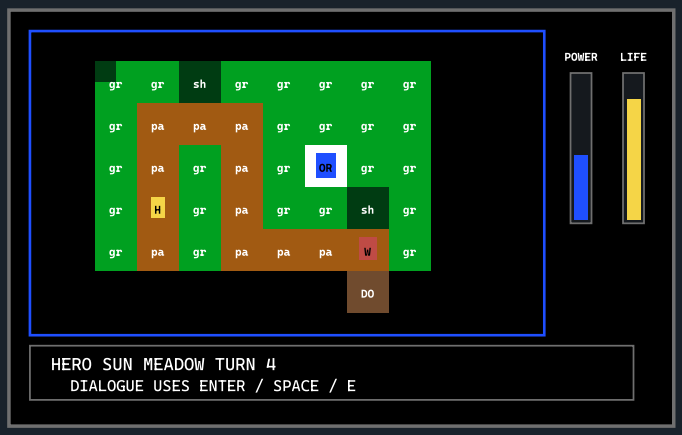
ACS Runtime: Classic Mode + Profiles

Open Editor

Visual Mode

Classic ACS

Sun Meadow (1, 1)



Objective

Speak to the Oracle, claim the Solar Seal, and return in classic ACS mode.

Controls

WASD / arrows move
E interacts or continues dialogue
Q inspects
Visual Mode switches presentation only.

State

Turn: 4
Flags: quest_started true
Inventory: Oracle Charm x1

Profile: life 12, power 4, speed 3

MILESTONE 15

ACS Construction Set MVP

Back To Play | Save Draft | Reset | Playtest

EDIT FLOW

- 1 Adventure
- 2 Atlas
- 3 Map
- 4 Libraries
- 5 Logic
- 6 Publish

ADVENTURE SETUP

Game Identity

Title

Oracle of the Solar Seal

Description

Adventure-wide metadata separate from map editing.

WORLD ATLAS

Regions & Maps

Adventure Package

- Regions
- Maps
- Tiles / Entities

Selected Map

Inner Shrine

Category + Region

interior | Shrine

Create Map

MAP WORKSPACE

Terrain, Entities & Local Objects

Layer Mode: Terrain | Tile Brush: floor | Place Definition: Shrine Wolf | Place

Active Tool: floor brush

stone	stone	stone	stone	stone	stone
stone	floor	floor	floor	floor	stone
stone	floor	floor	floor	altar	stone
stone	door	stone	stone	stone	stone

Selected map workspace keeps terrain, population, exits, and local trigger

DEPENDENCIES

Selected Map Relations

Region | Tile layer | Entity instances | Triggers + exits

LIBRARIES

Reusable Definition

Entity Definition

Shrine Wolf

Placement / Behavior

multiple | pursue

Profile + starting gear

life 6 | power 3 | speed 4

LOGIC & QUESTS

Rule Trigger

When | If | Then

Map reference + coordinate

map_shrine at 4,2

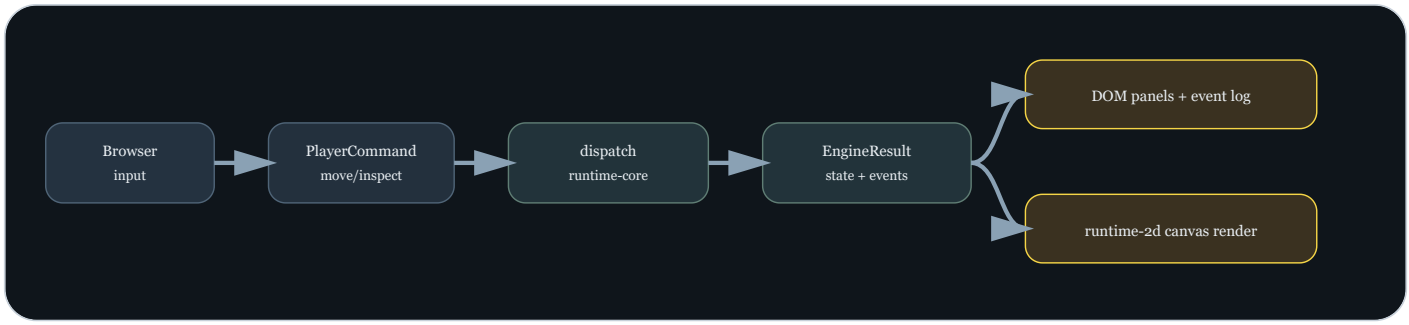
TEST & PUBLISH

Draft Health

Validate | Publish

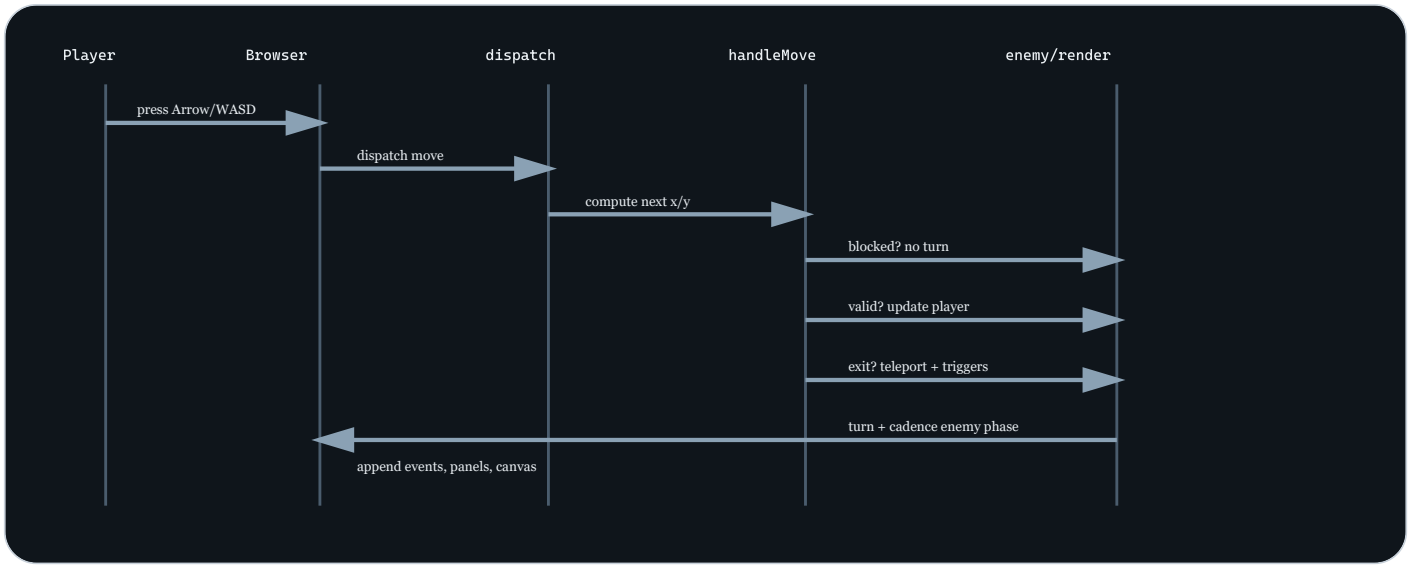
Validation and release workflow stay separate from content editing.

Runtime Input To Rendering



Runtime input follows a narrow path: `apps/web/src/index.ts` turns browser input into a `PlayerCommand`, `runtime-core` mutates session state and emits events, then the browser updates panels and asks `runtime-2d` to redraw.

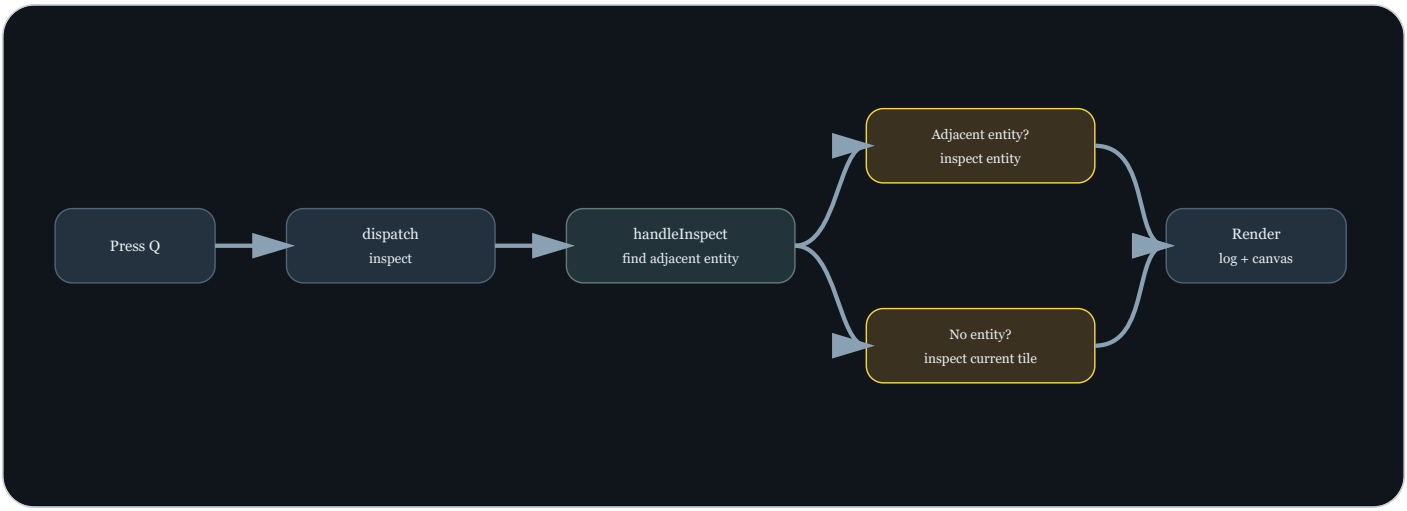
Use Case 1: Move Command



1. The keydown handler in `apps/web/src/index.ts` maps Arrow keys and `WASD` to `{ type: "move", direction }`.
2. `runCommand` calls `session.dispatch(...)`, appends returned events, and calls `renderEverything`.
3. `dispatch` calls `handleMove` and uses the returned boolean to decide whether the command consumed a turn.
4. `handleMove` computes the next coordinate, rejects bounds or occupied destinations, or updates `state.player`.
5. If the destination is an exit, the engine changes `currentMapId`, teleports the player, and runs map-load and tile triggers.
6. If movement succeeded, `dispatch` increments `state.turn`, emits `turnEnded`, and runs the enemy phase.
7. The browser updates map name, player position, flags, inventory, dialogue, event log, and canvas.

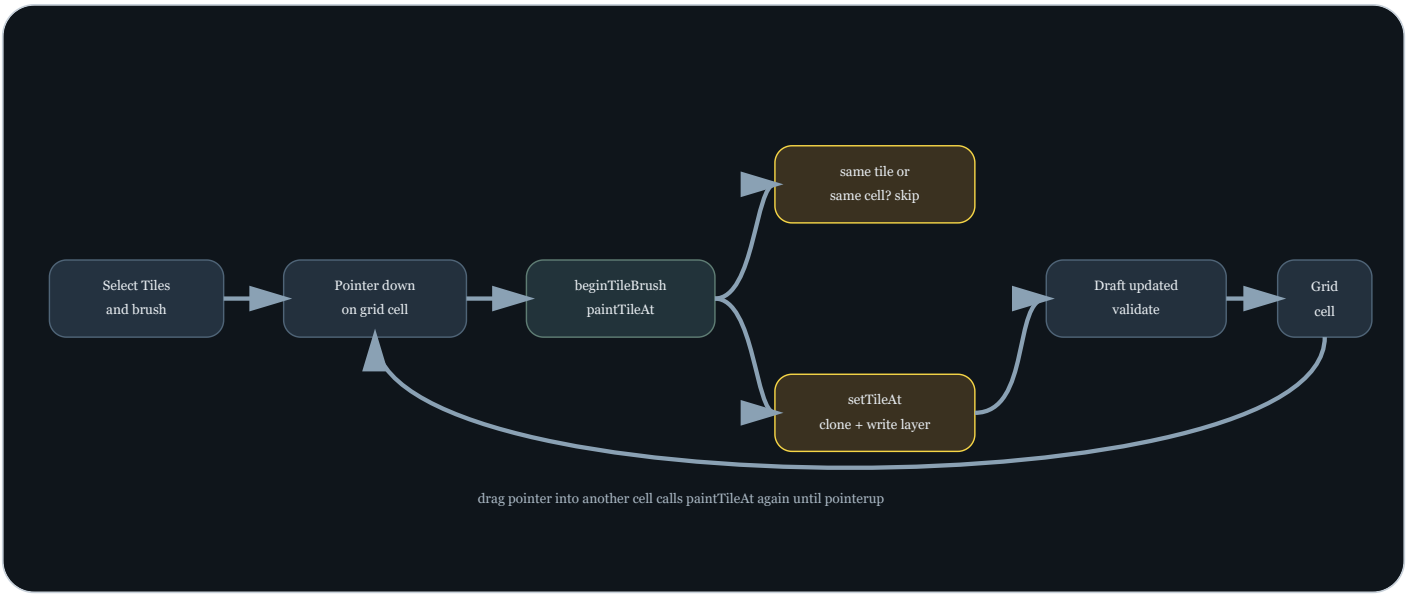
Current behavior: blocked movement does not consume a turn and does not grant enemies a free action. Enemy behavior profiles may define `turnInterval`; the sample Shrine Wolf uses `turnInterval: 3`, so it only acts on every third successful player turn.

Use Case 2: Inspect Command



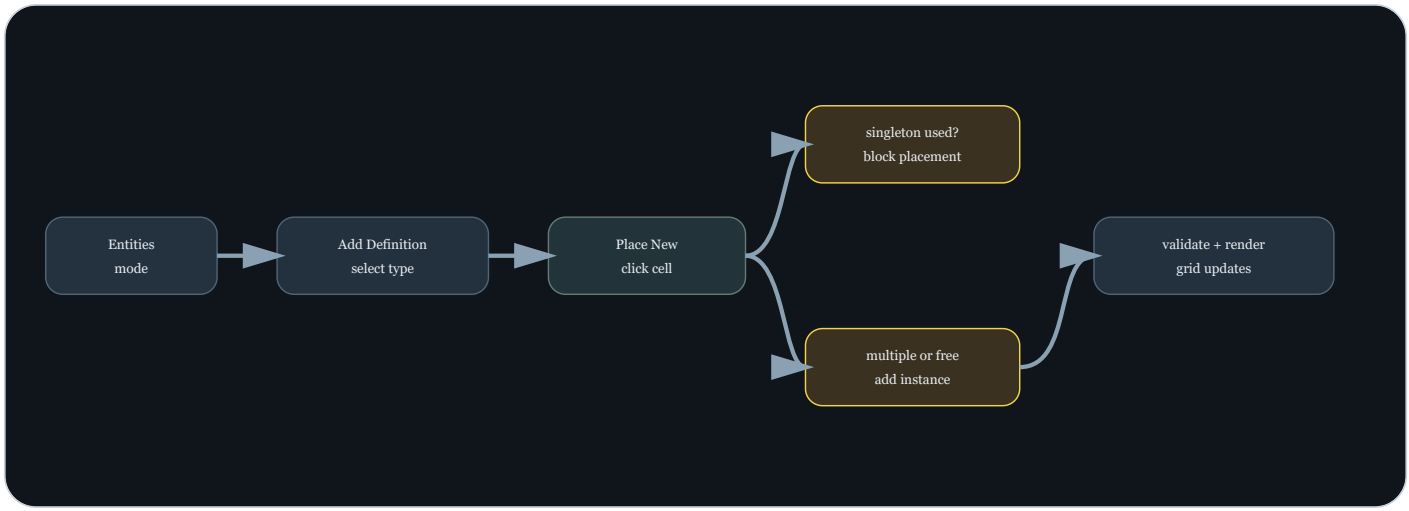
1. The browser maps `Q` to `{ type: "inspect" }`.
2. `dispatch` calls `handleInspect(undefined, events)` and treats a completed inspection as a turn-consuming action.
3. With no explicit direction, `findEntityInDirection` searches for the first active adjacent entity on the current map.
4. If an adjacent entity exists, the engine emits `inspectResult` for that entity.
5. If no adjacent entity exists, the engine emits an `inspectResult` describing the player's current coordinate and map id.
6. Inspect does not directly move the player, grant items, set flags, or start dialogue.
7. `dispatch` increments `state.turn`, emits `turnEnded`, then runs the enemy phase. Enemies still respect their configured `turnInterval`.

Use Case 3: Editor Tile Brush



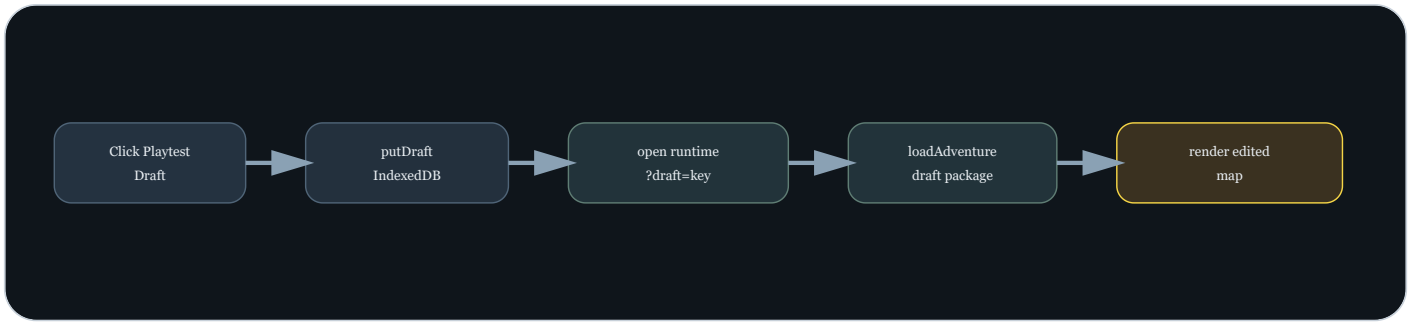
1. The designer selects `Tiles` mode and picks a tile id. The browser stores it in `selectedTileId`.
2. `renderBrushPreview` updates the visible active brush.
3. Each grid cell has a `pointerdown` handler in tile mode. Pressing a cell calls `beginTileBrush(x, y)`.
4. `beginTileBrush` sets `isTileBrushActive = true`, clears the duplicate-cell guard, and calls `paintTileAt`.
5. `paintTileAt` skips if the editor is not in tile mode, if the same cell was already painted in this drag, or if the cell already has the selected tile.
6. If the tile differs, the browser calls `setTileAt(draft, currentMapId, x, y, tileId)` from `editor-core`.
7. `setTileAt` clones the package, finds the map and active layer, calculates `index = y * map.width + x`, writes the tile id, and returns a new draft.
8. The browser refreshes the changed grid cell, reruns local validation, and updates project/publish controls.
9. Dragging into another cell calls `paintTileAt` through `pointerenter`. Pointer up ends the brush.

Use Case 4: Editor Entity Placement



1. The designer switches to `Entities` mode.
2. The editor shows `Move Entity`, `Add Definition`, and `Place New`.
3. Choosing a reusable definition stores `selectedEntityDefinitionId` and enters new-instance placement mode.
4. `singleton` definitions are blocked after one instance exists anywhere in the adventure.
5. `multiple` definitions can be placed repeatedly.
6. `addEntityInstance` clones the package, creates a unique id, pushes an `EntityInstance`, then validation and grid rendering rerun.

Editor To Playtest



Painting a tile changes only the editor draft. To see it in play, the editor saves the draft to IndexedDB and opens the runtime with a draft query parameter. The runtime loads that same `AdventurePackage`, so there is no separate editor-only map format.

Validation, Publishing, Save, And Load

Validation and publishing

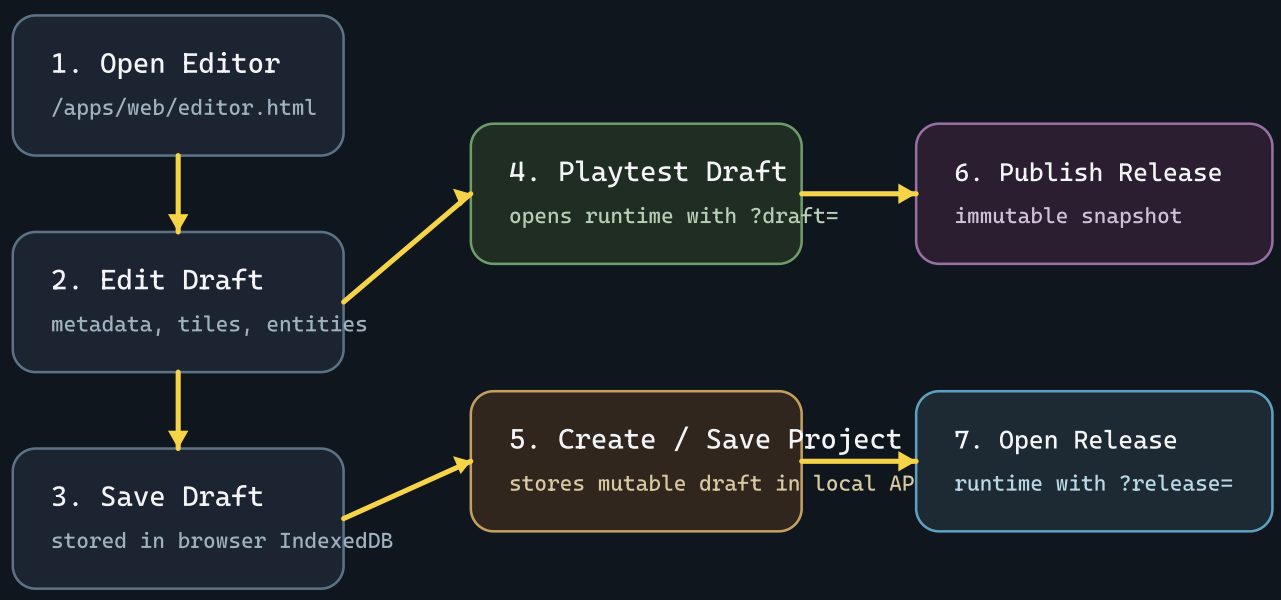
1. Local edits rerun `validateAdventure(draft)`.
2. `Validate Draft` calls the API validation endpoint.
3. Project save and publish are disabled when local validation is blocking.
4. The API validates again before creating an immutable release.

Runtime save/load

1. `Save` calls `session.serializeSnapshot()`.
2. `persistence.saveSession` stores the snapshot in IndexedDB.
3. `Load` retrieves the record and calls `engine.loadAdventure(activeAdventure, snapshot)`.
4. The browser rerenders from restored state.

ACS User Workflow

How the current application is used from editing to playtesting to publishing.

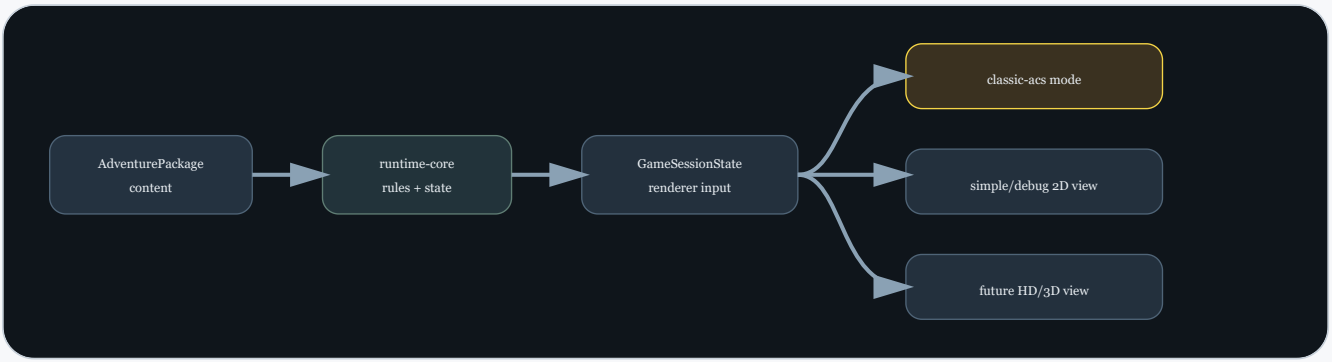


Forward Milestone Path: Classic ACS Feel

The project should deliberately capture the feel of the original 1980s Adventure Construction Set while keeping the engine free of renderer assumptions. The legacy image set points to two major tracks: a classic gameplay panel and deeper construction-set authoring tools.

Classic runtime presentation

The classic runtime mode should be a presentation layer over the same `AdventurePackage`, `RuntimeSnapshot`, and `GameSessionState` used by the current renderer.



- fixed-aspect vintage gameplay panel
- tile/icon map viewport on a dark field
- right-side status rail with clearly labeled life, power, and future actor-resource meters
- bottom message band for location names, prompts, scrollable Classic ACS dialogue, interaction text, and command hints
- pixelated sprite scaling and controlled palette
- asset IDs resolved through manifests so the same map can later render with HD or 3D assets

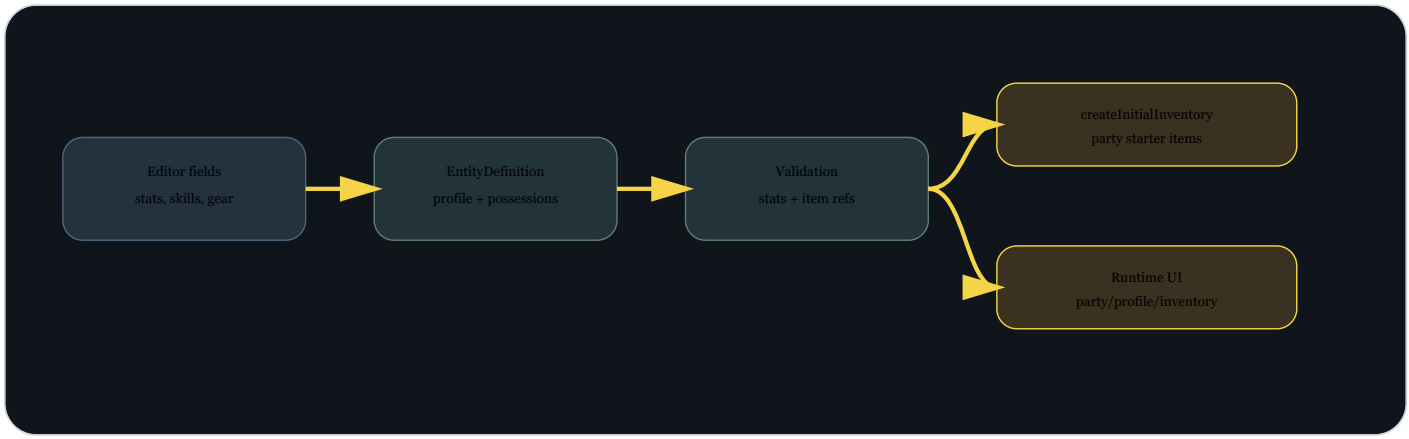
Classic editor capability track

The original editor suggests future modes for terrain pictures, thing pictures, creature pictures, map/floor wiring, reusable definitions, actor profiles, possessions, triggers, portals, text, and dialogue.

1. **Milestone 10:** completed `classic-acs` visual mode with a gameplay viewport, right status rail, bottom message band, and renderer theme switching.
2. **Milestone 11:** completed sprite manifests, a first classic asset set, entity sprite IDs, manifest-driven classic renderer lookup, and validation warnings for missing sprite references.
3. **Milestone 12:** completed entity definition editing for reusable metadata, placement policy, sprite asset ids, faction, and behavior tuning.
4. **Milestone 13:** completed dialogue text editing and structured trigger editing for existing records.
5. **Milestone 14:** completed map categories, current-map structure metadata editing, and blank map creation.
6. **Milestone 15:** completed richer entity profiles, skills, starting possessions, runtime party/profile/inventory rendering, editor fields, and validation checks.
7. **Milestone 16:** next, deepen no-code trigger/action construction and make item/profile conditions easier to author visually.

Milestone 15 Entity Profiles And Starting Gear

Milestone 15 enriches reusable `EntityDefinition` records. Definitions can now carry a `profile` with stat fields and skills, plus `startingPossessions` that seed the party inventory when a new runtime session starts.

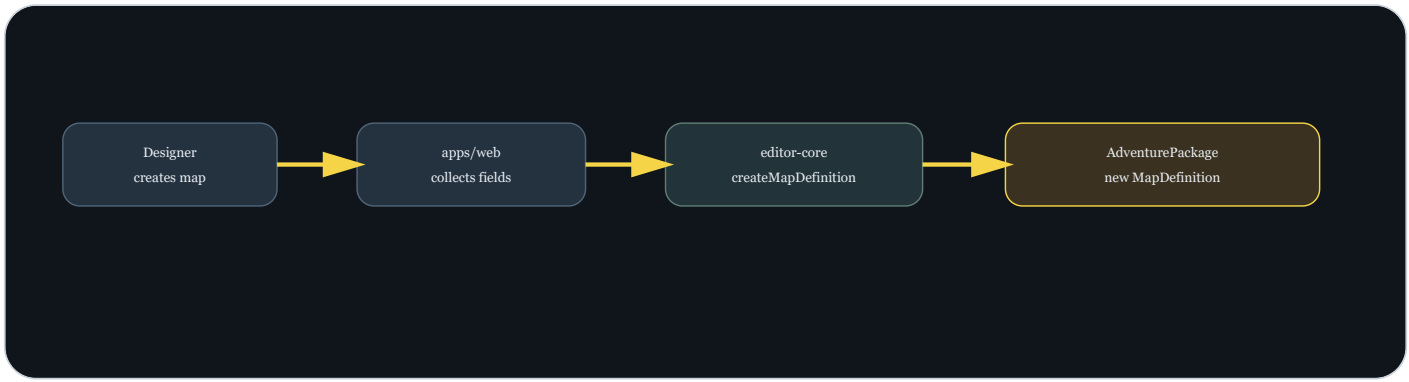


1. The browser editor reads definition form values from `apps/web/src/editor.ts`, including life, power, speed, skills, and starting possession text such as `item_oracle_charm:1`.
2. Those values are stored on the reusable `EntityDefinition`, not on each placed `EntityInstance`. Placement still remains map-specific.
3. `packages/validation` rejects negative/non-integer stats, unknown starting item ids, and invalid starting possession quantities.
4. When `runtime-core` creates a new session, `createInitialInventory` walks the party definition ids in `startState.party`, finds each definition, and folds its `startingPossessions` into inventory state.
5. The runtime browser UI displays the party, profile summary, and named inventory, while `runtime-2d` continues to render from state without owning these rules.

This keeps the future path open: combat can read stats, AI can read skills or faction, equipment can read starting possessions, and alternate renderers can present the same data differently.

Milestone 14 Map Structure Editing

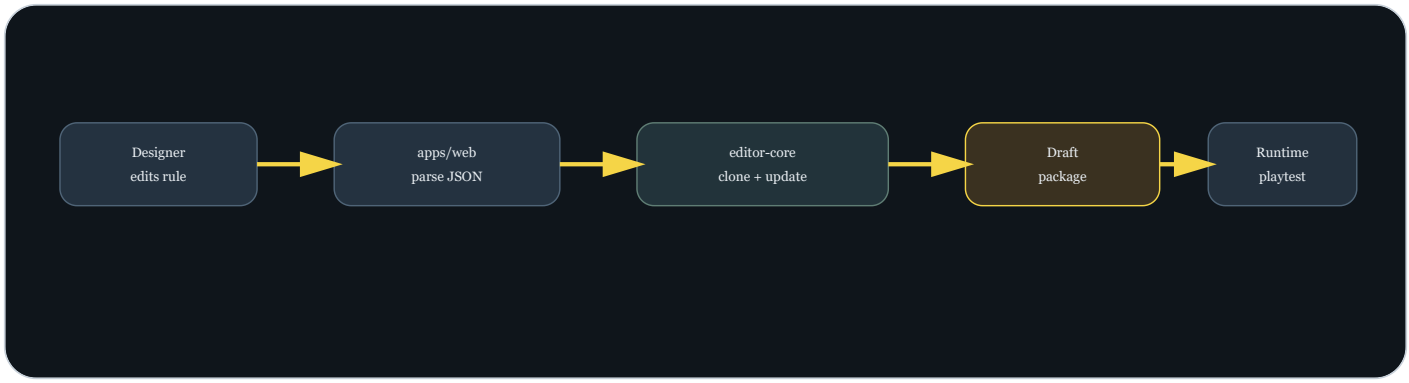
Milestone 14 begins the world-structure track. Maps now have optional category metadata: `world`, `region`, `local`, `interior`, and `dungeonFloor`. The editor can also create a new blank map with a base tile layer.



New maps start with one base tile layer, a repeated fill tile, no exits, and no placed entities. Exit/portal wiring and map deletion remain later milestones because they require careful reference-safety rules.

Milestone 13 Dialogue And Trigger Editing

Milestone 13 adds construction-set controls for existing dialogue records and structured trigger records. Browser controls gather intent, `editor-core` clones and updates the `AdventurePackage`, validation reruns, and playtest loads the same draft package.



Flow: Designer edits dialogue or trigger fields -> `apps/web/src/editor.ts` parses form values and JSON arrays -> `editor-core` updates the package copy -> validation reruns -> playtest loads the same draft.

Dialogue editing updates the first node speaker, text, and continue label. Trigger editing updates type, optional map/x/y location, run-once behavior, conditions JSON, and actions JSON. Invalid JSON is rejected by the editor before it can replace the trigger data.

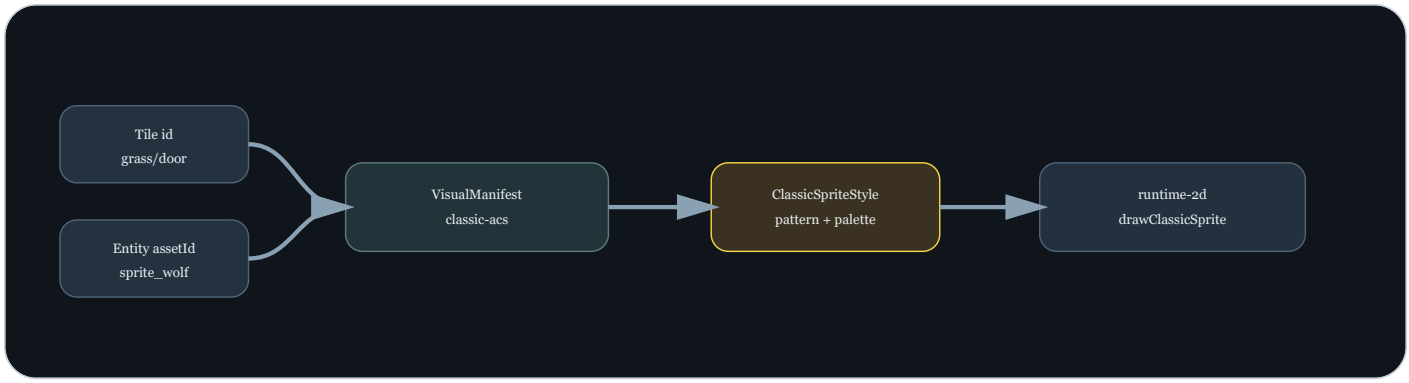
Current limitation: Milestone 13 edits existing dialogue and trigger records. Brand-new record creation, deletion, and a fully visual no-JSON rule builder remain future work.

Milestone 12 Entity Definition Editing

Milestone 12 added browser controls for reusable entity definition metadata. Existing placed entities remain separate `EntityInstance` records that point back to reusable `EntityDefinition` records by id.

Milestone 11 Sprite Manifests

Milestone 11 moves the classic visual mode from hardcoded tile and entity assumptions toward manifest-driven presentation data. `runtime-core` remains renderer-agnostic; `runtime-2d` reads the adventure's `classic-acs` visual manifest when deciding how to draw canvas sprites.

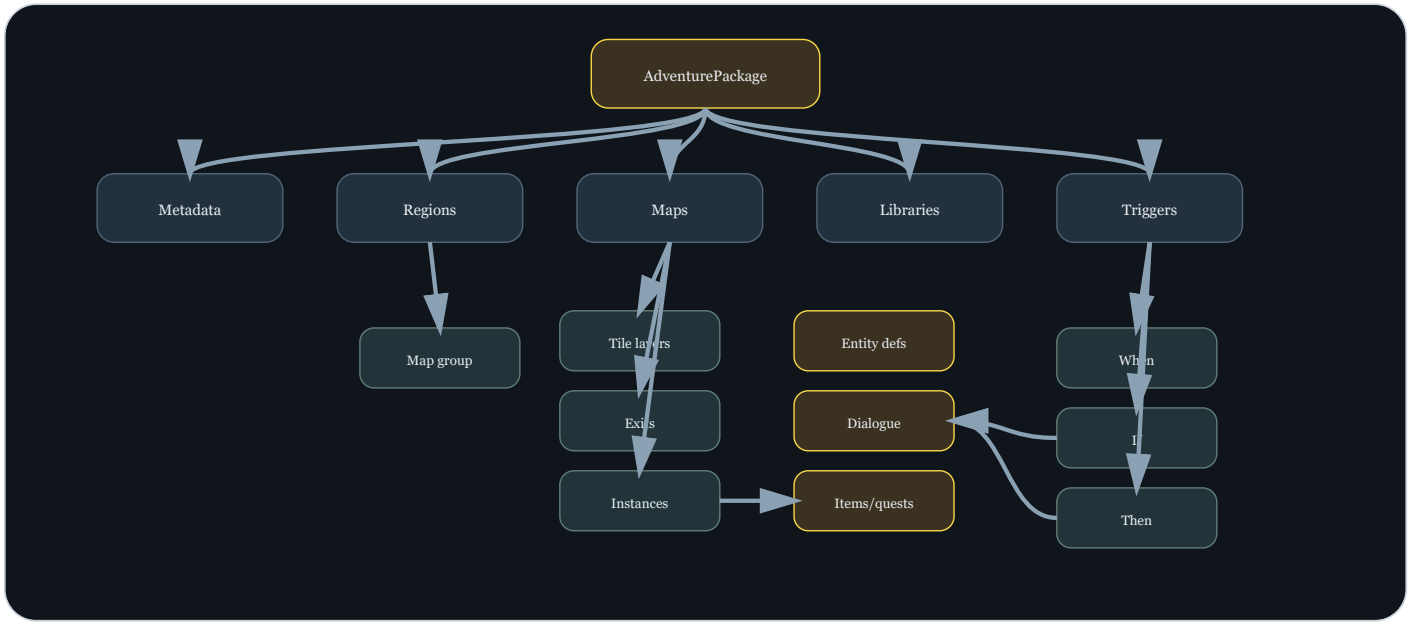


- `AdventurePackage.visualManifests` stores renderer-facing presentation metadata outside the simulation rules.
- The sample adventure defines `Classic Solar Seal Sprite Set` with tile entries for grass, path, shrub, stone, floor, altar, door, water, and void.
- The Hero, Oracle, and Shrine Wolf definitions now point at stable sprite IDs through `assetId`.
- The classic renderer resolves a tile/entity sprite from the manifest first, then falls back to built-in defaults for older packages.
- Validation warns when classic sprite references are missing without blocking the project, because fallbacks keep content playable.

Recommended Editor Information Architecture

The current editor works, but its panels are arranged in milestone order rather than authoring order. As the game data grows, the editor should be organized around the relationships in `AdventurePackage`: Adventure owns Regions, Regions group Maps, Maps contain tile layers, exits, placed entities, and local triggers, while reusable libraries provide definitions that maps and triggers reference.

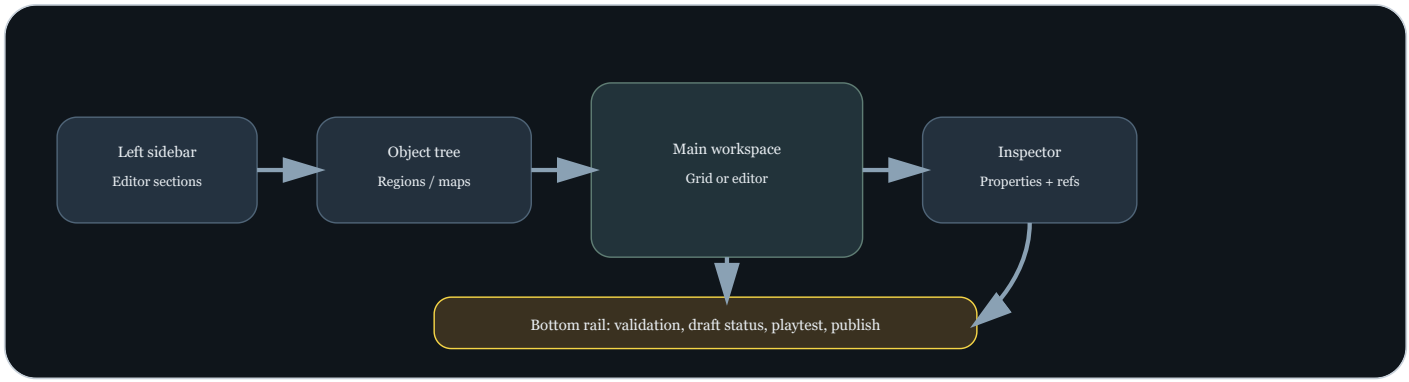
Core principle: show containment first, references second, and implementation details third. A designer should first understand where they are in the adventure, then what objects live there, then what definitions and rules those objects point to.



Proposed Editor Areas

Area	Purpose	Primary objects	Why it belongs here
Adventure Setup	Project-level identity and rules	Metadata, rules, start state, validation	These affect the whole adventure and should not be mixed into map editing.
World Atlas	Spatial structure	Regions, maps, map categories, exits, start position	This is the natural home for Milestone 14 map structure tools.
Map Workspace	Editing the selected place	Tile layers, entity instances, local triggers, exits	This is the main canvas-centric construction area.
Libraries	Reusable building blocks	Entity definitions, items, dialogue, quests, assets, visual manifests	These are referenced by maps, triggers, and instances.
Logic & Quests	Cross-map behavior	Triggers, conditions, actions, quest stages, dialogue links	Logic often connects many objects and needs reference-aware tools.
Test & Publish	Workflow actions	Validation, drafts, playtest, projects, releases	These are workflow controls, not content objects.

Recommended Editor Layout



1. The left sidebar chooses the authoring area: Adventure, World Atlas, Map Workspace, Libraries, Logic, or Test & Publish.
2. The middle column shows the selected area's object tree, such as Regions with nested Maps.
3. The main workspace shows the selected map grid, definition editor, or trigger flow.
4. The right inspector shows properties and references for the selected object.
5. The bottom rail always shows validation status, unsaved draft status, and playtest/publish actions.

Object-Specific Flows

World Atlas

Shows Regions first, then Maps inside each Region. Selecting a map exposes category, dimensions, region assignment, exits, incoming references, and whether the start state points there.

Map Workspace

Centers on a selected map with tabs for Terrain, Entities, Exits, Triggers, and Preview. Selecting a cell should reveal tile id, occupant, exit, and triggers attached to that coordinate.

Libraries

Holds reusable definitions: entity definitions, item definitions, dialogue, quests, assets, and visual manifests. Each library item should show where it is used.

Logic & Quests

Turns triggers into a `When / If / Then` builder. This can later replace JSON-first editing while still emitting the same domain `TriggerDefinition` data.

Dependency Panel Rule

Every selected object should have a dependency panel. This is the most important UI principle for making relationships understandable.

Selected object	Dependency panel should show
Region	Maps inside the region and any region-level notes/source references.
Map	Region, exits, entities, local triggers, incoming links, and start-state usage.
Map cell	Tile id, occupant, exit, and triggers attached to that coordinate.
Entity definition	All placed instances using that definition and any asset reference.
Entity instance	Definition, map location, placement constraints, and local triggers if applicable.
Dialogue	Triggers/actions that call the dialogue record.
Trigger	Every map, dialogue, item, quest, tile, or entity reference used by its conditions/actions.

Implemented Edit Game Layout

The browser editor now implements the first version of this organization without changing the underlying data model. The same editor functions still update `AdventurePackage`, but the screen is arranged by information hierarchy.

- Left navigation: the authoring flow from Adventure Setup through Test & Publish.
- Context column: Adventure Setup and World Atlas for map selection, region assignment, category metadata, and blank-map creation.
- Center workspace: selected-map editing for terrain and entity instances.
- Inspector column: dependencies, reusable libraries, dialogue, trigger logic, validation, and publishing.

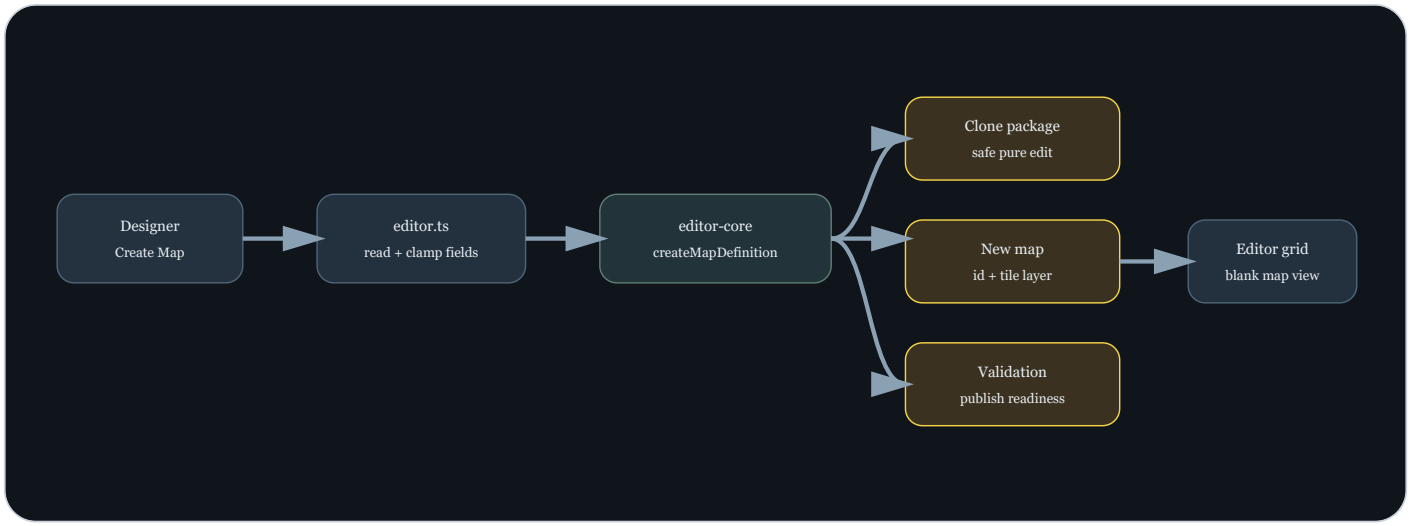
Later milestones can deepen each area with a real region tree, selected-cell inspector, exit graph, reference backlinks, and a structured trigger builder.

Current Feature Implementation Map

Feature	Browser entry	Core owner	Output
Runtime movement	`apps/web/src/index.ts` keydown	`runtime-core.dispatch` and `handleMove`	State, events, enemy phase, canvas redraw
Inspect/interact/dialogue	`Q`, `E`, dialogue advance keys, Classic scroll arrows	`runtime-core` inspect, interact, dialogue helpers	Event log, Classic message band or Debug dialogue panel, optional state changes
Runtime save/load	`Save` and `Load` buttons	`RuntimeSnapshot` plus `persistence`	IndexedDB snapshot restored into the engine
Visual modes	`Visual Mode` dropdown	`runtime-2d`	Same state rendered as Classic ACS or Debug Grid
Tile brush	Editor grid pointer events	`editor-core.setTileAt`	Map tile layer update, validation, grid refresh
Entity placement/movement	Editor entity mode controls	`editor-core.addEntityInstance` and `moveEntityInstance`	Entity instances update; singleton rules enforced

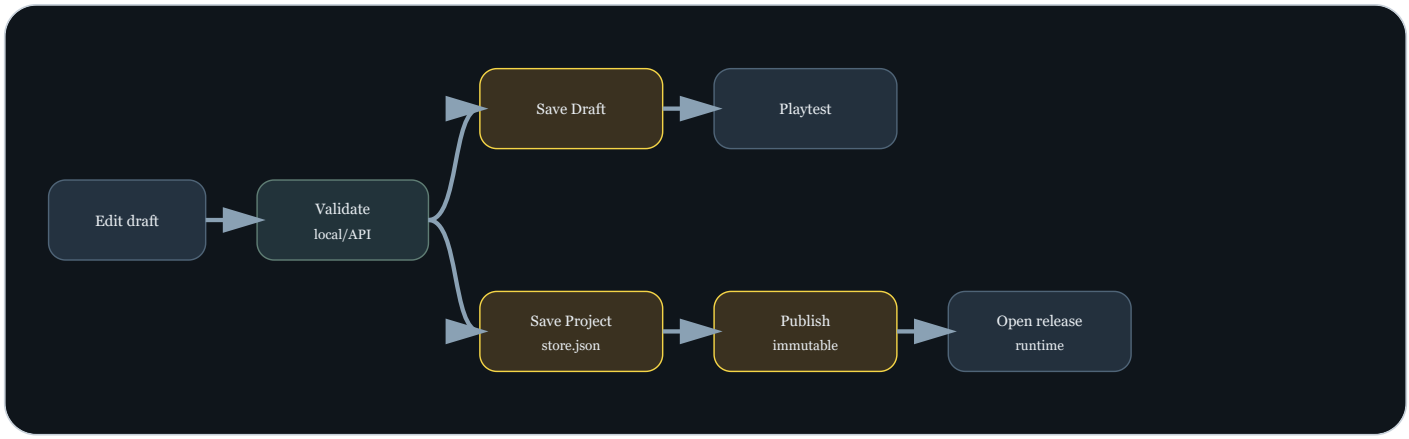
Definition editing	`Entity Definition` panel	`editor-core.updateEntityDefinition`	Reusable definition changes affect placed instances
Dialogue/trigger editing	`Dialogue Text` and `Rule Trigger` panels	`editor-core.updateDialogueNode` and `updateTriggerDefinition`	Runtime playtest uses edited text/rules
Map structure and creation	`World Structure` panel	`editor-core.updateMapDefinition` and `createMapDefinition`	Map metadata edits and new blank maps
Projects/releases	Project panel buttons	`project-api`, `apps/api`, `validation`	Project drafts and immutable releases in `store.json`

Use Case: Create A Milestone 14 Blank Map



1. The designer enters name, category, optional region, dimensions, and fill tile in the `World Structure` panel.
2. `createBlankMapFromEditor()` trims text, clamps dimensions, and calls `createMapDefinition(...)`.
3. `editor-core` clones the draft package, creates a unique map id, creates one base tile layer, fills it, and appends the map.
4. The browser switches `currentMapId` to the created map, reruns validation, and renders the new grid.
5. The map has no exits yet; reference-safe exit/portal wiring is intentionally future work.

Use Case: Full Authoring Loop



The editor updates an `AdventurePackage`. The runtime, validator, API, and release system all consume that same package shape, which avoids creating a separate editor-only format.

Documentation Generation Requirements

- After every milestone, update the User Guide tutorial so it tries every feature currently available.
- Explicitly highlight the latest milestone's features in the User Guide and System Reference.
- Keep Mermaid diagrams in Markdown and readable rendered equivalents in HTML/PDF.
- Include current runtime/editor screenshots or screenshot-style graphics where they help explain behavior.
- Document each major feature with an end-to-end flow from browser input to core operation to validation, persistence, or rendering output.
- Avoid broken image links, HTML scroll artifacts, overlapping diagram text, and page splits through important diagrams or callout boxes.

Extension Points

- A higher-resolution renderer can live beside `runtime-2d` as long as it consumes `AdventurePackage` and `GameSessionState`.
- A future 3D renderer should not change browser input or editor logic unless the content model needs richer spatial metadata.
- Real-time play would most likely evolve the command scheduling, enemy `turnInterval`, and enemy phase in `runtime-core`.
- Richer enemy AI belongs in `runtime-core` or a future AI package, not in the DOM or canvas renderer.
- New editor creation tools should start as pure functions in `editor-core`, then be wired into `apps/web/src/editor.ts`.